

Pulse — Real-Time Stock Market Dashboard

A comprehensive engineering and theoretical guide to the full-stack architecture, stateless JWT security, quantitative indicator algorithms, real-time RAG pipeline with Google Gemini, and automated background schedulers.

CORE BACKEND

Java 17, Spring Boot 3.2.4

FRONTEND SPA

React 19, Vite 8, Chart.js

DATABASE

PostgreSQL (Neon) +
Flyway

AI ENGINE

Google Gemini 3.5 Flash
(SSE)

1. System Architecture & The Request Lifecycle

Pulse is engineered as a decoupled, full-stack financial monitoring platform designed for high data density, low-latency market analysis, and zero-hallucination research. The architecture enforces strict separation of concerns across a 6-tier backend hierarchy.



How to Explain This to an Interviewer:

"My backend is structured around Spring Boot 3 layered architecture. Every request passes through a custom `JwtAuthFilter` that supports both HTTP Bearer headers and query parameters (for SSE streams). Valid requests are routed to controllers, which delegate to domain services. Market telemetry is retrieved directly from Yahoo Finance, relational data is persisted in serverless PostgreSQL via Spring Data JPA, and long-running research streams execute asynchronously via Spring's `@Async` thread pools."

2. Authentication, Authorization & Password Recovery Flow

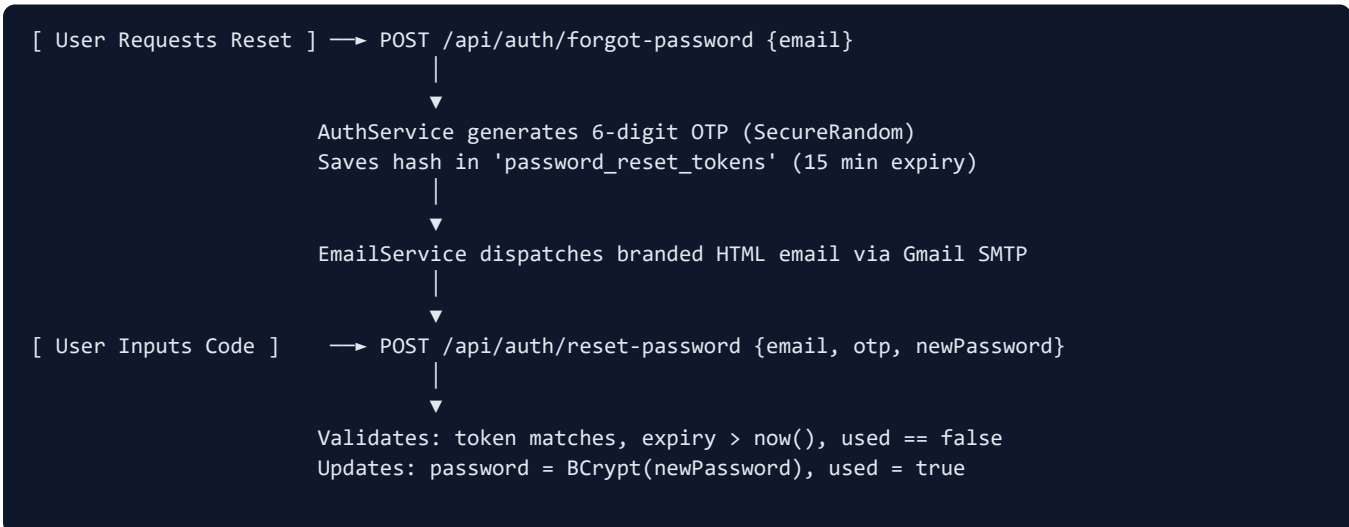
Subsystem 1 Stateless JSON Web Token (JWT) Security Architecture

Pulse avoids server-side session memory overhead by employing a 100% stateless JWT token model. No session state or cookies are stored on the server, permitting horizontal backend scaling.

- **Password Hashing:** Passwords are never stored in plaintext. They are salted and encrypted using `BCryptPasswordEncoder` (work factor 10) in `SecurityConfig.java`.
- **Token Lifecycle:** Upon successful login, `JwtService.java` generates a compact HMAC-SHA256 signed token. The payload contains the user's email (Subject), issued timestamp (iat), and 24-hour expiration window (`86,400,000 ms`).
- **Dual-Vector Token Interception:** Standard browser `EventSource` APIs (used for streaming AI research) cannot send custom `Authorization` request headers. To solve this, `JwtAuthFilter.java` inspects:
 1. Header: `Authorization: Bearer <token>` (used by standard fetch REST requests).
 2. Query Parameter: `?token=<jwt>` (used exclusively by native EventSource SSE connections).
- **Spring Security Context:** Once validated, a `UsernamePasswordAuthenticationToken` is bound to `SecurityContextHolder.getContext()`, making the authenticated user accessible in any downstream controller via `Authentication authentication`.

Subsystem 2 Forgot Password Flow via 6-Digit Cryptographic OTP

To eliminate insecure reset links, Pulse uses a time-limited 6-digit One-Time Password (OTP) verification flow:



3. Real-Time Telemetry & Canvas Sparkline Visualizations

Rather than relying on mock datasets, Pulse directly interfaces with Yahoo Finance REST endpoints to provide real-time equity quotes and intraday historical series.

Endpoint	External Provider URL	Data Extracted & Business Processing
GET <code>/api/stocks/search?q=...</code>	<code>query2.finance.yahoo.com/v1/finance/search</code>	Live equity and ETF autocomplete. Filters out bonds and options; returns clean symbol, name, and exchange.
GET <code>/api/stocks/quote?symbol=...</code>	<code>query1.finance.yahoo.com/v8/finance/chart/{symbol}</code>	Parses regular market price, day change, % change, volume, 52-week high, and 52-week low.
GET <code>/api/stocks/chart?symbol=...&range=1d</code>	<code>query1.finance.yahoo.com/v8/finance/chart/{symbol}?interval=5m</code>	Pulls 5-minute intraday timestamps and closing prices. Formats parallel arrays for Chart.js canvas rendering.
POST <code>/api/stocks/tracked</code>	Internal PostgreSQL (Neon DB)	Persists stock in user's watchlist with a composite database unique constraint <code>(user_id, symbol)</code> .

Interview Tip (Chart.js Sparklines):

"In the React frontend, I used Chart.js drawing directly to an HTML5 Canvas rather than heavyweight SVG charts (like D3 or TradingView). When polling 10+ stocks simultaneously, canvas rendering avoids creating hundreds of DOM elements. The sparkline dynamically samples intraday bars and color-codes the stroke: **Emerald Green (#10B981)** for net gains, or **Crimson Red (#EF4444)** for intraday drops."

4. The Pulse Quantitative Engine: Mathematical Theory

Unlike standard text applications, Pulse contains a built-in algorithmic math engine implemented in `QuantitativeEngineService.java`. It takes raw 30-day historical closing price series and calculates deterministic mathematical indicators on the fly:

A. Simple Moving Average (SMA-20 & SMA-50)

The unweighted mean of the previous N daily closing prices:

$$\text{SMA}_N = \frac{1}{N} \sum_{i=1}^N P_i \quad \text{where } N \in \{20, 50\}$$

Trend Regime Classification:

- $\text{Price} > \text{SMA}_{20} \ \&\& \ \text{SMA}_{20} \geq \text{SMA}_{50} \rightarrow \text{STRONG_BULLISH}$
- $\text{Price} > \text{SMA}_{20} \ \&\& \ \text{SMA}_{20} < \text{SMA}_{50} \rightarrow \text{MODERATE_BULLISH}$
- $\text{Price} < \text{SMA}_{20} \ \&\& \ \text{SMA}_{20} \leq \text{SMA}_{50} \rightarrow \text{STRONG_BEARISH}$
- $\text{Price} < \text{SMA}_{20} \ \&\& \ \text{SMA}_{20} > \text{SMA}_{50} \rightarrow \text{MODERATE_BEARISH}$

B. 14-Day Relative Strength Index (RSI-14)

Measures the speed and change of price movements to identify overbought or oversold market conditions:

$$\text{RS} = \frac{\text{Average Gain over 14 Days}}{\text{Average Loss over 14 Days}}, \quad \text{RSI} = 100 - \frac{100}{1 + \text{RS}}$$

Momentum Regime:

- $\text{RSI} \geq 70.0 \rightarrow \text{OVERBOUGHT}$ (Elevated mean-reversion risk)
- $\text{RSI} \leq 30.0 \rightarrow \text{OVERSOLD}$ (Potential bounce or capitulation)
- $30.0 < \text{RSI} < 70.0 \rightarrow \text{NEUTRAL MOMENTUM}$

C. 30-Day Realized Annualized Volatility

Measures historical statistical price dispersion using continuous log-returns scaled by standard exchange trading days (252 days):

$$r_i = \ln\left(\frac{P_i}{P_{i-1}}\right), \quad \sigma_{\text{daily}} = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (r_i - \bar{r})^2}, \quad \sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252} \times 100\%$$

Volatilities above 35% are classified as `ELEVATED_VOLATILITY`, alerting users to price risk.

D. 52-Week Range Position

Calculates where the current price sits relative to its 1-year trading envelope:

$$\text{Range Position (\%)} = \left(\frac{\text{Current Price} - \text{52W Low}}{\text{52W High} - \text{52W Low}} \right) \times 100$$

5. The RAG Pipeline & AI Assistant (Exclusively Google Gemini)

The Problem RAG Solves in Pulse:

Standard LLMs (like ChatGPT or standalone Gemini) fail in finance because equity prices change every second and training cutoff dates cause hallucinations. Pulse implements a **Real-Time Factual Grounding RAG Pipeline**. It acts as an open-book exam: the backend gathers live exchange facts first, attaches strict anti-hallucination directives, and requires Gemini to cite source document IDs (e.g. [\[DOC-1\]](#)).

The 5-Stage RAG Execution Flow



Why No Vector Database Was Used (Crucial Interview Answer):

"Interviewers often ask: 'Where is your Vector DB like Pinecone or Chroma?' My answer: Vector databases are designed for

static, unstructured text (like searching company PDFs). In stock markets, prices change every second; vector embeddings would become stale immediately. Furthermore, calculating an RSI or a 50-day moving average requires **exact deterministic mathematics**, not fuzzy semantic approximation. Therefore, real-time API retrieval and on-the-fly math calculation is the architecturally superior design."

6. Automated Background Scheduler & Alert Notification Engine

Pulse features a resilient background worker implemented in `AlertScheduler.java` and `EmailService.java` that automates price threshold monitoring without requiring any user browser to be open.

```
Spring @Scheduled Cron: "0 */2 3-10 * * MON-FRI" (Market Hours)
```



```
1. FETCH ACTIVE ALERTS  
priceAlertRepository.findByTriggeredFalse()
```



```
2. DEDUPLICATE REQUESTS (GROUP BY SYMBOL)  
alerts.stream().collect(Collectors.groupingBy(PriceAlert::getSymbol))  
-> 50 users tracking NVDA results in only 1 Yahoo API call, not 50!
```



```
3. EVALUATE THRESHOLD BREACH  
• Condition ABOVE: currentPrice >= targetPrice  
• Condition BELOW: currentPrice <= targetPrice
```



```
4. DISPATCH EMAIL & UPDATE DATABASE (ANTI-SPAM GUARANTEE)  
• alert.setTriggered(true) & commit to PostgreSQL immediately.  
• EmailService dispatches responsive HTML email via Gmail SMTP:587.
```

Key Engineering Highlights of the Scheduler:

- **Zero Rate-Limit Exhaustion:** By grouping alerts by stock symbol via Java Streams, the scheduler makes $O(N_{\text{unique symbols}})$ HTTP calls instead of $O(N_{\text{total alerts}})$.
- **State Machine & Idempotency:** Setting `triggered = true` in PostgreSQL before dispatching notifications guarantees that users receive **exactly one email alert** when their target is reached, rather than an email every 2 minutes.
- **Non-Blocking Thread Architecture:** Spring Boot's scheduling pool runs on a distinct background worker thread, ensuring heavy batch checks never block Tomcat HTTP request threads.

7. Database Schema & Flyway Versioning

All database tables in PostgreSQL (Neon Cloud) are managed via version-controlled SQL migrations located in `backend/src/main/resources/db/migration/` :

Version	Migration File	Table Created	Schema Constraints & Details
V1	V1__Create_Users_Table.sql	users	id BIGSERIAL PRIMARY KEY, email VARCHAR UNIQUE NOT NULL, password VARCHAR NOT NULL, created_at TIMESTAMP.
V2	V2__Create_Tracked_Stocks_Table.sql	tracked_stocks	Foreign key to users(id) ON DELETE CASCADE. Composite unique index: UNIQUE(user_id, symbol).
V3	V3__Create_Price_Alerts_Table.sql	price_alerts	Stores symbol, target_price, condition (ABOVE/BELOW), triggered BOOLEAN DEFAULT FALSE.
V4	V4__Create_Password_Reset_Tokens.sql	password_reset_tokens	Stores 6-digit cryptographic OTP, expiry_date TIMESTAMP, used BOOLEAN DEFAULT FALSE.

8. Top 15 Technical Interview Questions & Answers

Q1: Why did you choose stateless JWT over HTTP Session cookies?

"Session cookies store session state in server RAM, which breaks horizontal scalability when running multiple instances behind a load balancer without sticky sessions or distributed Redis stores. JWT is stateless: the server signs the token using HMAC-SHA256, and all state (user identity, expiration) lives inside the token itself, reducing server memory to zero."

Q2: How does your backend handle Server-Sent Events (SSE) authentication?

"Standard browser `EventSource` does not allow custom headers like `Authorization: Bearer`. I engineered `JwtAuthFilter` to extract JWT tokens from both the `Authorization` header and the `?token=...` query parameter. It validates the token signature, sets up Spring's `SecurityContext`, and permits authenticated SSE streaming."

Q3: Why Server-Sent Events (SSE) instead of WebSockets for the research assistant?

"WebSockets are bidirectional, requiring custom protocol framing, handshake upgrades, and continuous connection state. AI token streaming is strictly **unidirectional** (server pushing text tokens to client). SSE runs natively over standard HTTP/1.1 or HTTP/2, includes built-in browser reconnection, passes through enterprise proxies, and is much lighter on memory."

Q4: Why did you not use a Vector Database for your RAG engine?

"Vector DBs are built for static text embeddings. Financial market data changes every second—embeddings would become stale immediately. Furthermore, calculating technical indicators (SMA, RSI, Volatility) requires exact deterministic math, not approximate cosine similarity. Pulse retrieves live exchange data on-demand and computes indicators in Java, guaranteeing zero staleness and zero math hallucinations."

Q5: How do you prevent LLMs from hallucinating stock prices?

"Through strict prompt engineering and citation anchoring. The prompt instructs Google Gemini to formulate its analysis exclusively using the retrieved document cards (`[DOC-1]`, `[DOC-2]`) and mandates that every figure must cite its source. In the React UI, citations are interactive badges that link directly to the raw, verifiable telemetry card."

Q6: What happens if Google Gemini is down or the user has no API key?

"I implemented an offline fallback: the **Pulse Grounded Quant Engine**. In `LlmResearchService.java`, if no key is present or if the Gemini API call throws an exception, the backend algorithmically synthesizes the exact same report in Java with simulated token streaming cadence. The user never experiences an error screen."

Q7: How did you optimize Yahoo Finance API calls in your cron scheduler?

"If 100 users set price alerts on NVDA, making 100 API calls every 2 minutes would trigger rate limits. I used Java Streams to group untriggered alerts by ticker symbol:

`alerts.stream().collect(Collectors.groupingBy(PriceAlert::getSymbol))` . The scheduler makes **exactly 1 API call per unique ticker**, regardless of how many users are tracking it."

Q8: How do you ensure users aren't spammed with alert emails every 2 minutes?

"The `price_alerts` table includes a `triggered` boolean flag. When the scheduler detects that `LastPrice` has crossed `targetPrice`, it sets `triggered = true` and saves the record in PostgreSQL before sending the email. Future cron cycles filter for `findByTriggeredFalse()`, guaranteeing exactly one notification per price breach."

Q9: How do you prevent SQL injection in your backend?

"By using Spring Data JPA and Hibernate ORM. All repository queries use parameterized SQL prepared statements under the hood. User input is never concatenated directly into raw SQL strings."

Q10: Why did you use Flyway instead of Hibernate's `ddl-auto=update`?

"`hibernate.ddl-auto=update` is dangerous for production because it can silently alter columns, cause data loss, and does not version-control schema changes. Flyway enforces reproducible, versioned SQL migration scripts (`v1` through `v4`) that execute sequentially on application boot."

Q11: How do you handle password storage securely?

"I use `BCryptPasswordEncoder` with a work factor of 10. BCrypt incorporates a random 128-bit salt and an adaptive key derivation function that resists rainbow-table and brute-force GPU attacks."

Q12: How is natural language entity resolution handled in the research assistant?

"If a user asks 'Analyze Apple and Tesla', the regex extracts 'Apple' and 'Tesla'. The backend's `resolveToTicker()` method first checks if the word is an exact ticker. If not, it dynamically queries Yahoo Finance's search endpoint to resolve company names to tickers ('AAPL', 'TSLA') and caches the result in a thread-safe `ConcurrentHashMap` ."

Q13: How is CORS handled between Vite (port 5173) and Spring Boot (port 8080)?

"Configured globally in `WebConfig.java` via `WebMvcConfigurer.addCorsMappings()`, allowing specified origin patterns, methods (GET, POST, PUT, DELETE, OPTIONS), and credentials. In production on Render/Vercel, origin headers are strictly matched."

Q14: How is global error handling implemented?

"Using a centralized `@RestControllerAdvice` class (`GlobalExceptionHandler.java`). It intercepts Spring validation exceptions (`MethodArgumentNotValidException`), authentication failures (`InvalidCredentialsException`), and business exceptions, translating them into consistent JSON error envelopes with timestamp, status code, and message."

Q15: How is your Spring Boot backend optimized for cloud container deployment?

"It is containerized with a multi-stage Dockerfile targeting Eclipse Temurin 17 JRE. I tuned JVM memory flags using `-Xmx256m` and `SerialGC`, enabled GZIP compression for JSON payloads (`server.compression.enabled=true`), and capped HikariCP connection pool size to 3 to run efficiently inside free-tier cloud containers on Render."